CS2201 Computer Organization & Architecture

L11: Control Hazards & Branch Prediction



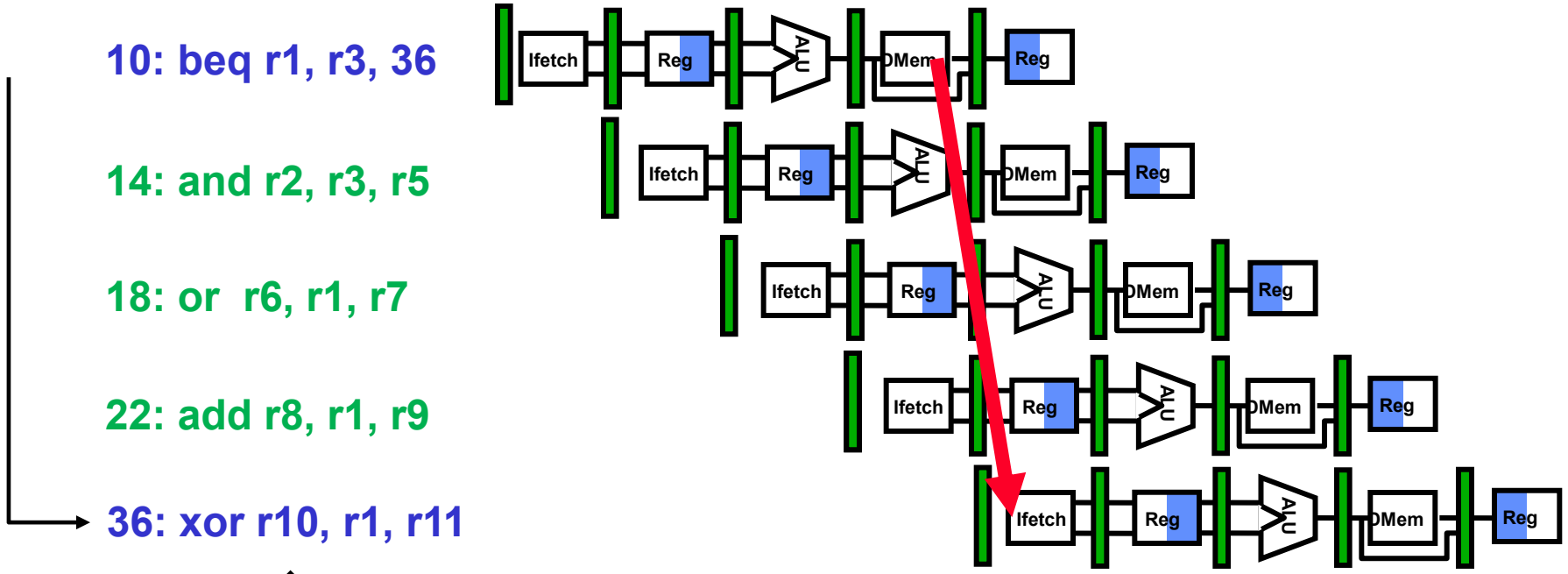
John Jose

Associate Professor

Department of Computer Science & Engineering

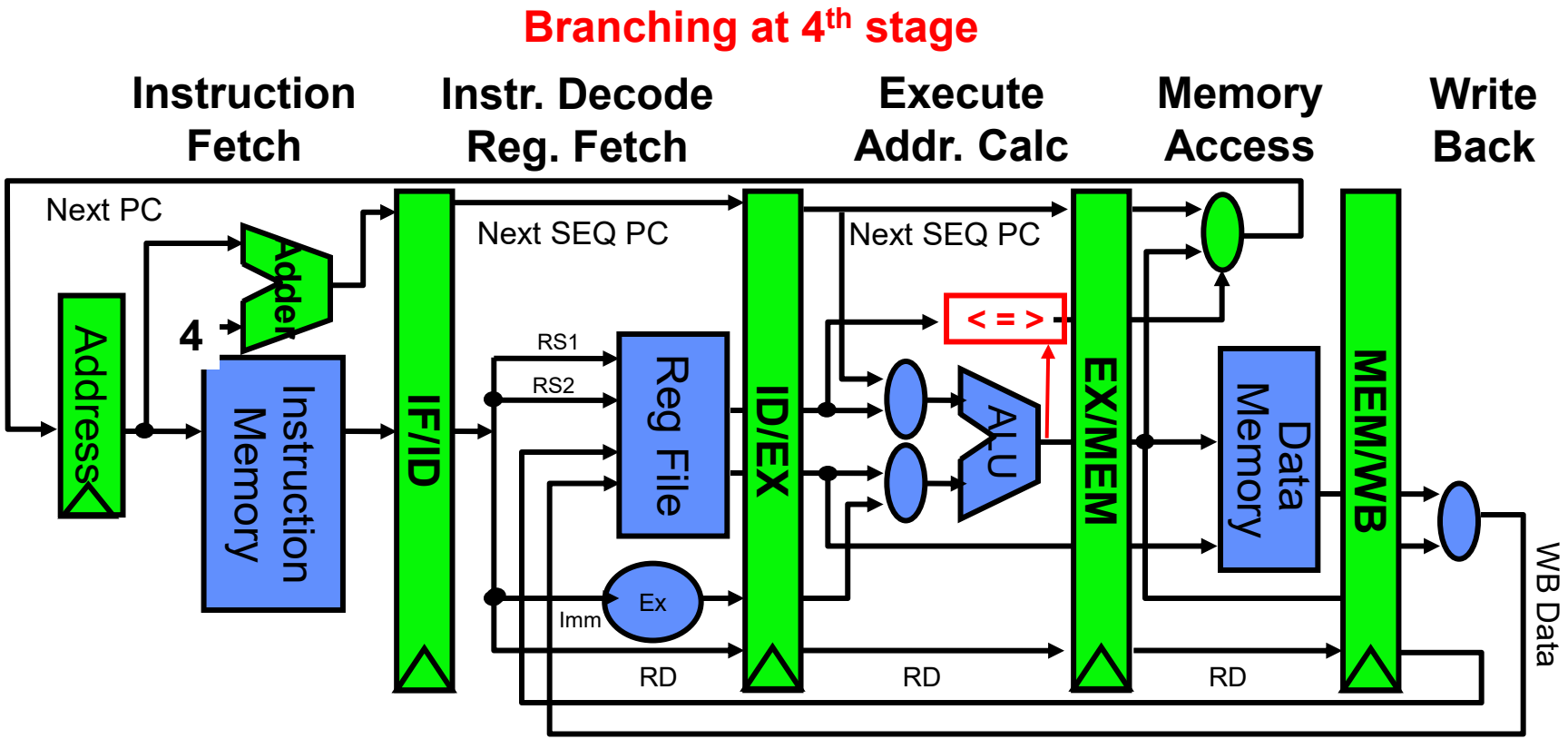
Indian Institute of Technology Guwahati

Control Hazard on Branches



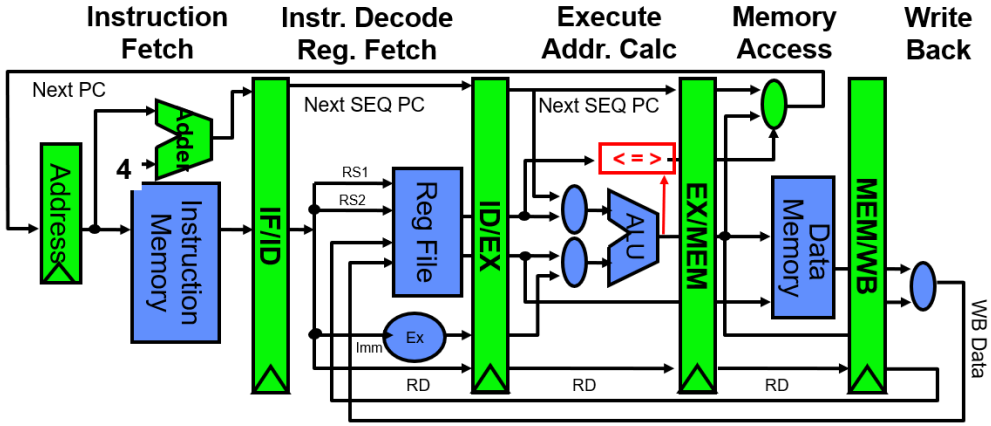
- Three Stage Stall
- Branch resolution in CC4.
- 3 instructions already in the pipeline

Conventional MIPS Pipeline



Conventional MIPS Pipeline

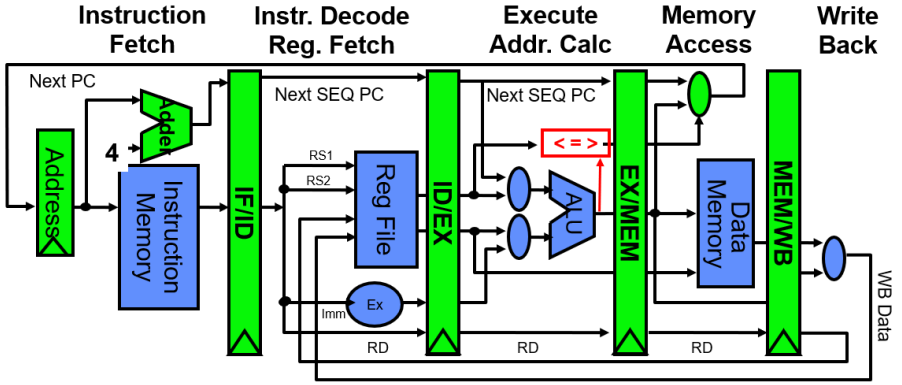
- The target address and branch outcome are finalized after the EX stage (computed by the ALU) and updated in MEM stage.

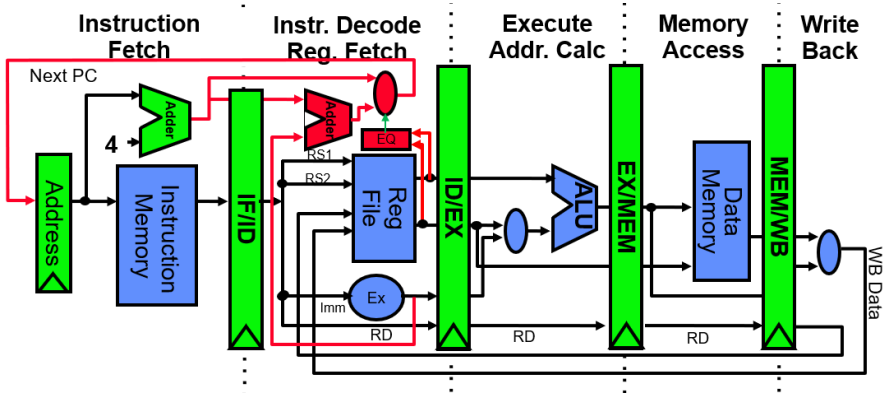


- If the branch is taken, 3 instructions already fetched into the pipeline (IF, ID, and EX) must be flushed (3 cycle penalty)
- Simple hardware design → reuses existing ALU for condition check
- Flushing 3 stages on every taken branch incurs a heavy penalty

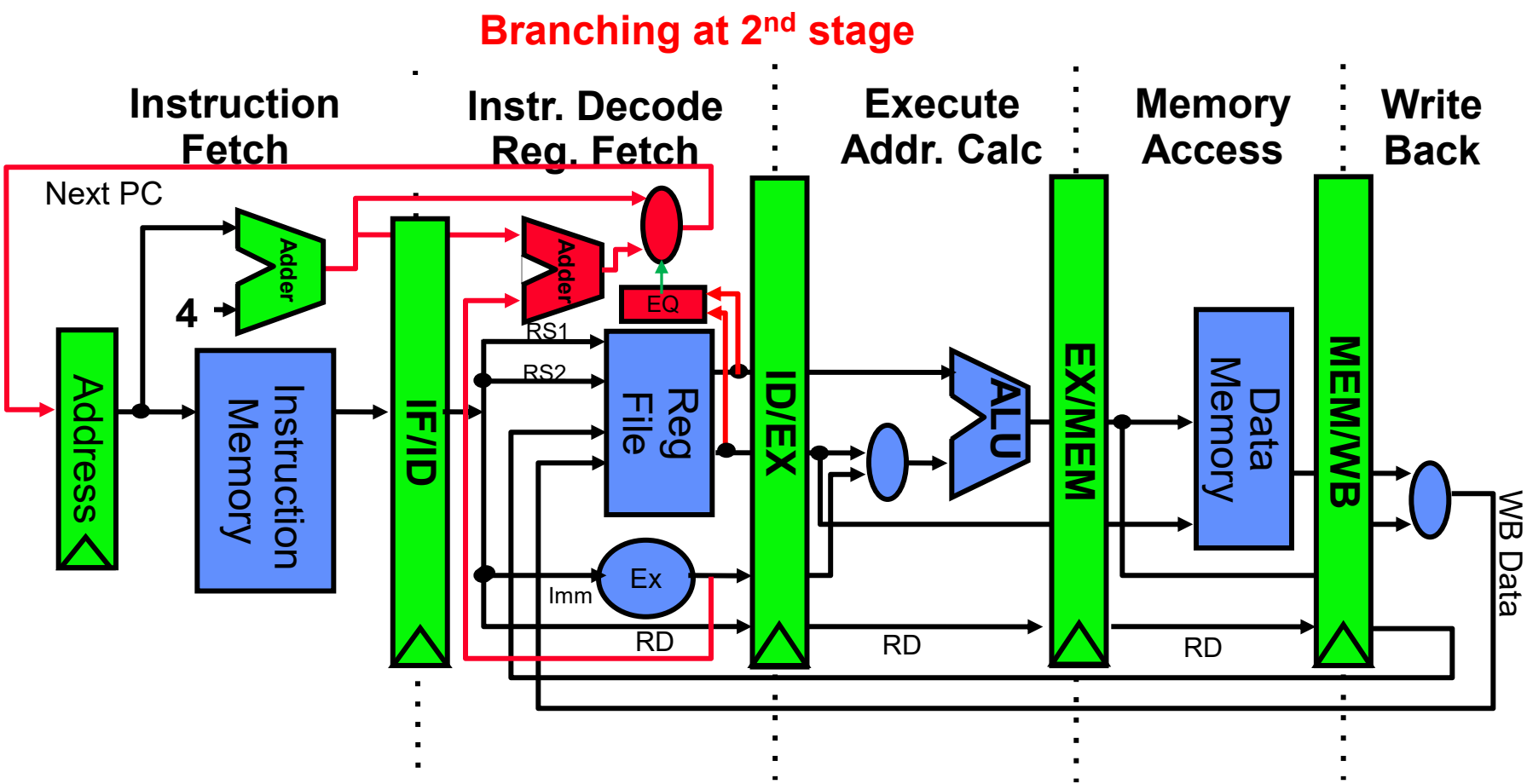
How to reduce branch penalty?

- The target address adder and condition check logic (comparator) to be moved to ID stage.
- Branch instructions in ISA limited to simple comparisons: BEQ, BNE
- If the branch is taken, 1 instruction (IF stage) needs to be flushed.
- Creates severe data hazards if the branch condition depends on an arithmetic instruction immediately preceding it.





Branch Optimized MIPS Pipeline



Four Branch Hazard Alternatives

1. Stall until branch direction is clear
2. Predict Branch Not Taken
3. Predict Branch Taken
4. Delayed Branch

Four Branch Hazard Alternatives

1: Stall until branch direction is clear

2: Predict Branch Not Taken **BNE R1, R2, 200** **PC=PC+196**

- Execute successor instructions in sequence
- Squash instructions in pipeline if branch actually taken

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	**idle**	**idle**	**idle**	**idle**			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Four Branch Hazard Alternatives

3: Predict Branch Taken **BNE R1, R2, 200** **PC=PC+196**

- But branch target address in is not known by IF stage
- Target is known at same time as branch outcome (ID)
- Incurs 1 cycle branch penalty

Four Branch Hazard Alternatives

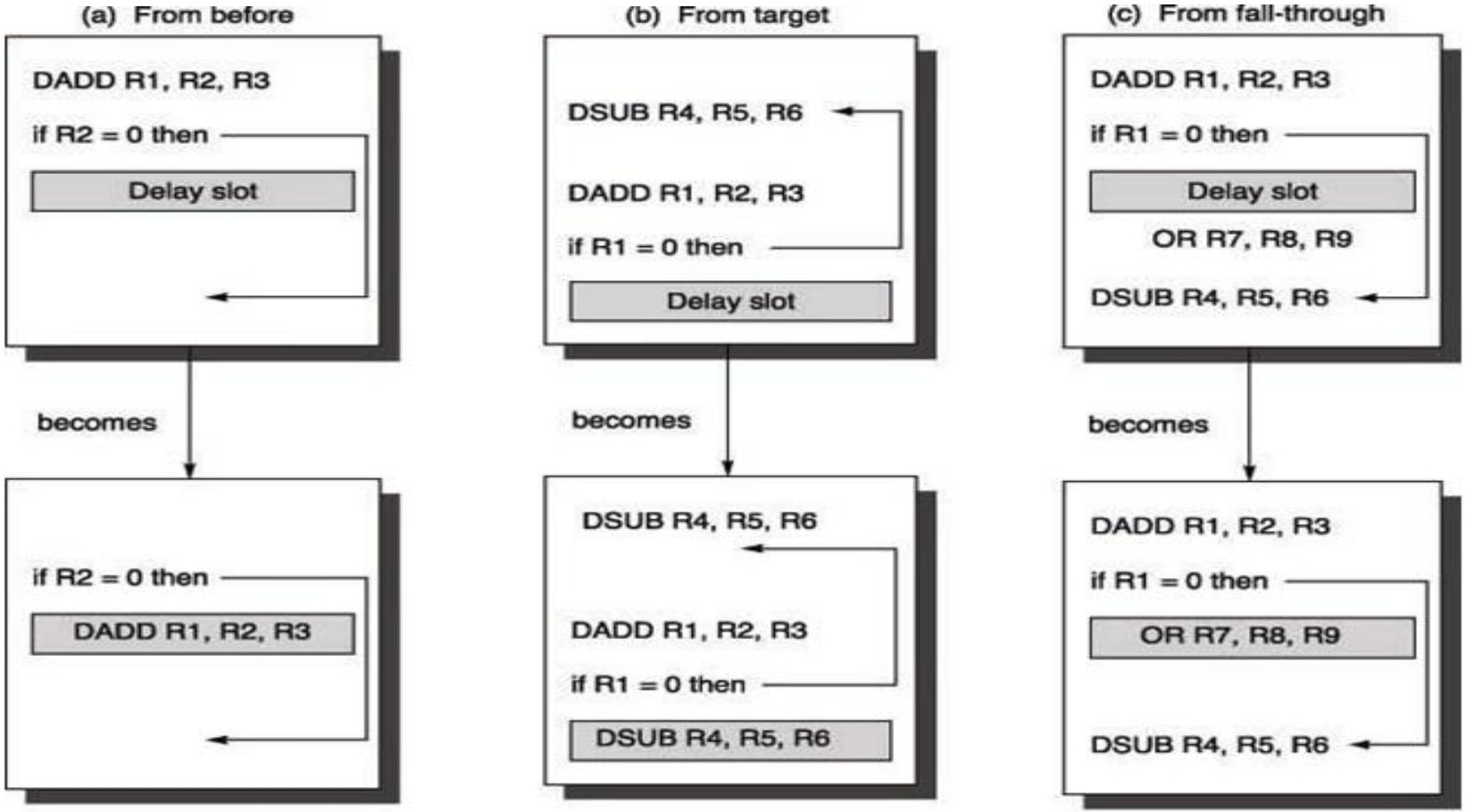
4: Delayed Branch

- Define branch to take place AFTER one instruction following the branch instruction.
- 1 slot delay allows proper decision and branch target address in 5 stage pipeline
- **Where to get instructions to fill branch delay slot?**

Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Filling Branch Delay Slot



Conditional Branches

- When do you know you have a branch?
  - During ID cycle (Could you know before that?)
- When do you know if the branch is Taken or Not-Taken
  - During EXE cycle/ ID stage depending on the design
- We need for sophisticated solutions for following cases
  - Modern pipelines are deep (10 + stages)
  - Several instructions issued/cycle
  - Several predicted branches in-flight at the same time

Dynamic branch prediction

- Execution of a branch requires knowledge of:
  - Branch instruction - encode whether instruction is a branch or not. Decide on taken or not taken (i.e., prediction can be done at IF stage)
  - Whether the branch is Taken/Not-Taken (hence a branch prediction mechanism)
  - If the branch is taken what is the target address (can be computed but can also be “precomputed”, i.e., stored in some table)
  - If the branch is taken what is the instruction at the branch target address (saves the fetch cycle for that instruction)

Dynamic branch prediction

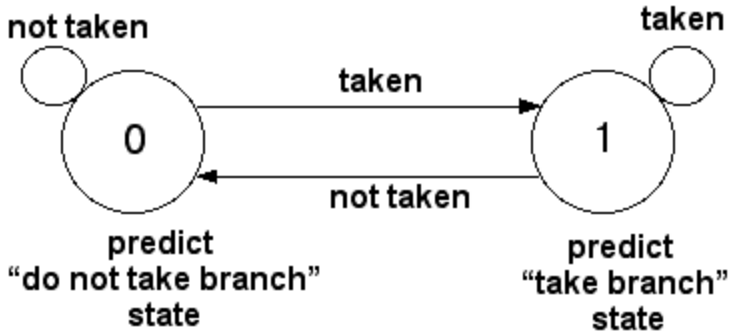
- Use a Branch Prediction Buffer (BPB)
  - Also called Branch Prediction Table (BPT), Branch History Table (BHT)
  - Records previous outcomes of the branch instruction.
  - Used to predict the likely outcome of future branches based on this history
- A prediction using BPB is attempted when the branch instruction is fetched (IF stage or equivalent)
- It is acted upon during ID stage (when we know we have a branch)

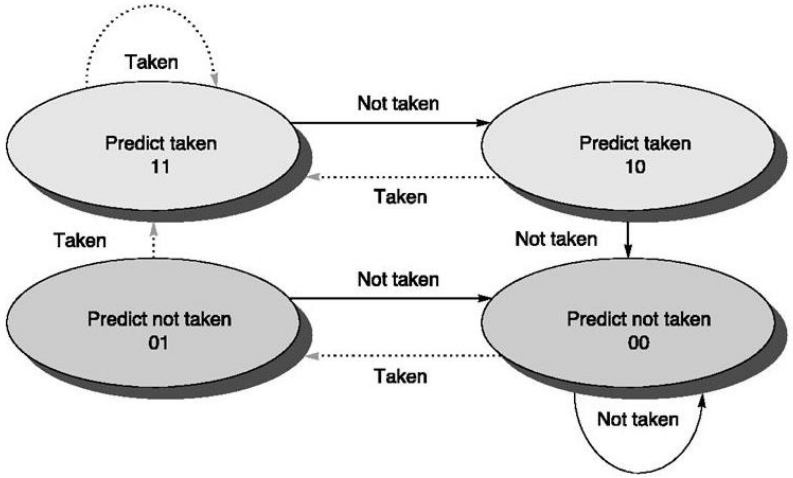
Dynamic branch prediction

- Has a prediction been made (Y/N)
  - If not use default “Not Taken”
- Is it correct or incorrect ?
- Two cases:
  - Case 1: Yes and the prediction was correct (known at ID stage) or No but the default was correct: No delay
  - Case 2: Yes and the prediction was incorrect or No and the default was incorrect: Delay

Prediction Scheme with 1 or 2 bit FSM

- The use of a 2-bit predictor will allow branches that favor taken (or not taken) to be mispredicted less often than the one-bit case. (reinforcement learning)





Branch Prediction In Hardware

- Branch prediction is extremely useful in loops.
- A simple branch prediction can be implemented using a small amount of memory indexed by lower order bits of the address of the branch instruction**. (branch prediction buffer)**
- One bit stores whether the branch was taken or not.
- The next time the branch instruction is fetched refer this bit

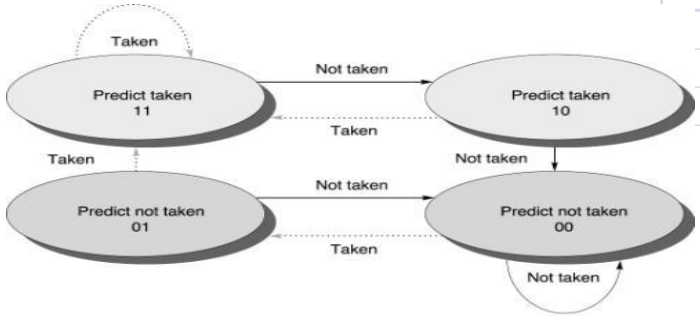
Advanced Branch Prediction Techniques

- **Correlating predictor: (m, n)**
  - Multiple n-bit predictors for each branch
  - One for each possible combination of outcomes of preceding *m* branches

```
if ( x = = 2)  /* br-1*/
       x = 0;
if ( y = = 2)  /* br-2*/
       y = 0;
if ( x != y)          /* br-3/
                do this
else do that
```

Branch Prediction - Example

Consider the last 5 actual outcomes of a single static branch. T means branch is taken and N means not taken. {oldest→ T T N N T ← latest}
A two level (1,2) type correlating branch predictor is used. Last branch outcome is used to index to the BHT which is initialized to 00 and 11 for N and T, respectively. How many mis-predictions are there and which of the branches in this sequence would be mis-predicted? Fill up the table for 5 branch outcomes.



Sl.No	Last Outcome	BHT N/T	Prediction	Outcome	Mis-Pre Y/N ?
1	N (initial)	00 / 11	N	T	YES
2	T	01 / 11	T	T	NO
3	T	01 / 11	T	N	YES
4	N	01 / 10	N	N	NO
5	N	00 / 10	N	T	YES
		01 / 10			



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/